

Q1. What are the pain points in using LLMs

A1. Large Language Models (LLMs) are incredibly powerful, but they're far from perfect. They can hallucinate, making up information that sounds convincing but isn't true, and there's still a big gap between their surface-level reasoning and the deeper, logical way humans think—especially when it comes to math or step-by-step problem solving. They also tend to slow down and lose focus with long prompts, since handling a lot of context is still tricky. On top of that, they can be sensitive to wording, giving very different answers to slightly different questions, and their responses aren't always consistent. Finally, running them takes a lot of computing power and cost, and they often carry forward biases from the data they were trained on.

Q2. Any surprises? Eg different conclusions from LLMs?

A2. One of the most notable surprises was that Claude provided a more nuanced analysis of the academic aspects of the project, including stakeholders, stakeholder conflicts, and specific use cases. For instance, Claude identified students, professors, and teaching assistants as key stakeholders and detailed use cases centered around showcasing the project. In contrast, the other large language models (LLMs) focused almost exclusively on the business side of the project.

Another surprising discovery was the stubbornness of some LLMs, particularly the free versions, when using zero-shot prompting. When prompted without any initial context, these models would often refuse to significantly modify their answers, even when I later provided additional information. This highlighted the importance of careful and context-rich prompting from the beginning to guide the model toward the desired outcome.

Q3. What worked best?

A3. The use cases from Gemini were more appropriate due to their comprehensive, formal structure. Each use case was systematically broken down into Preconditions, Main Flow, Subflows, and Alternative Flows. This meticulous approach provides a clear, unambiguous blueprint for developers, ensuring that all requirements and system behaviors are documented. The inclusion of a detailed stakeholder analysis further enhanced the output by providing essential business context and highlighting potential conflicts. This demonstrated a deeper understanding of the system's architecture and operational requirements.

Q4. What worked worst?

A4. The Llama output was less effective due to its lack of a formal structure. The use cases were presented as a simple, unstructured list, lacking the critical details required for a

proper software engineering document. The absence of preconditions and explicit flows leads to ambiguity and requires a significant amount of interpretation. Furthermore, several use cases were redundant, demonstrating a need for consolidation and refinement.

Q5. What pre-post processing was useful for structuring the import prompts, then summarizing the output?

A5. We pre-processed the LLMs with project requirements and deliverables beforehand to customize our responses to a semester-long project. Before we scaled up the project using a RAG system, we fed the previous use cases to the LLM to train it to generate new ones and to ensure none were repeated. A mix of careful prompting and one-shot prompting helped us retrieve refined results from the LLMs.

Although intensive pre-processing was used, we were not satisfied with the results from the LLMs. A lot of post-processing was required after certain responses. For example, while creating the use case flow diagram, the LLM left out many preconditions that needed to be included. We resolved this with additional prompts, pointing out the missing information to the LLM. We also did a lot of post-processing during Project 1c1, where we had to refine the 48 new use cases into about 15 MVP use cases. The use cases the LLM refined were generic at best, and to satisfy other actors in our MVP project, we had to add an additional use case that the team fed directly to the LLM.

Q6. Did you find any best/worst prompting strategies?

A6. The best strategy for us was careful prompting, where we gave clear structure, context, and expectations (like specifying use case sections: Preconditions, Main Flow, Subflows, Alternative Flows). This consistently yielded high-quality, specific outputs. The worst strategy was zero-shot prompting, where the prompt was just a vague instruction. The results were generic, incomplete, and require multiple revisions. Thus, careful prompting provided answers that had the structure, tone and detail we wanted, whereas zero-shot prompting resulted in answers that were generic and required fine-tuning.